Cuando usar cada tecnología?

- Cuando usar cada tecnología?
  - XML-RPC
  - SOAP
  - REST
  - GraphQL
  - gRPC
  - SOAP vs REST
  - REST vs GraphQL
  - HTTP Request, Long Polling , SSE, WebSockets, WebHooks

Tecnología	Cuando usar?
XML-RPC	Hoy en dia ya no se recomienda, incluso plataformas como Wordpress promueven desactivarlo
SOAP	Hoy en día ha disminuido, pero aplicaciones empresariales, gubernamentales, sector financiero y muchos sistemas antiguos, lo prefieren aún por su nivel de seguridad y el soporte de transacciones
REST	Cuando se requiere compatibilidad, facilidad de uso e implementación
Graphql	Cuando se requiere flexibilidad en la obtención de los datos
gRPC	Cuando se requiere alto desempeño, eficiencia y comunicación bidireccional
HTTP Request, Long Polling SSE, WebSockets, WebHooks	Soluciones rápidas para intercambios de datos simple entre un cliente y un servidor

XML-RPC



Tecnología desarrollada por Microsoft en la década de los 90 que ganó gran aceptación ya que promovía el uso de los estándares HTTP para comunicación, XML para transmitir los datos.

Cuando se debe usar

- Ya no se recomienda su uso.
- SOAP es la evolución de XML-RPC.

SOAP



Soap es una evolución de Xml-RPC. Tanto cliente como servidor intercambian información con mensajes en XML, debido a esto la cantidad y el tamaño de los mensajes que se tienen que enviar es muy grande, además de que su implementación requiere un amplio conocimiento de programación. Soap agrega WSDL *Web Service Description* language para describir el servicio proporcionado.

Cuando se debe usar

- Si lo que se requiere es alta seguridad y transacciones.
- Aplicaciones empresariales criticas.
- Entidades gubernamentales
- Sector financiero (banca, bolsa)

REST



Es una arquitectura que surgió como una alternativa a Soap, su propósito es compartir recursos mediante URLs.
Los datos generalmente se transmiten en formato Json (mucho mas ligero que Xml), aunque puede ser cualquier otro.
Usa los protocolos Http o Https para transmitir los datos, los verbos Http para indicar la acción sobre el recurso
y los códigos de estado de Http pueden identificar el resultado en la respuesta.

Cuando se debe usar

- Para Apis públicas.
- Solo se requieren solicitudes y respuestas simples. Y no se tienen requisitos detallados o muy específicos.
- No se necesita guardar el estado entre las solicitudes.
- Si se desea que los recursos sean fáciles de consumir.
- Cuando se tiene la libertad de escoger la arquitectura.
- Si los desarrolladores desconocen como implementar una API, Rest es de fácil aprendizaje.

Implementaciones

Javascript	Python
ExpressJs / Fastify / NestJs	FastAPI

GraphQL



Es un lenguaje de consulta desarrollado por Facebook mas flexible que Rest ya que
le permite al cliente obtener solo los datos que necesita.

Surgió como una alternativa a REST, aquí el servidor expone una estructura o esquema de los datos
y el cliente puede hacer consultas a ese esquema en un lenguaje de consultas que define GraphQL.
La ventaja respecto a REST es que aquí solo obtenemos lo que nos interesa.

Cuando se debe usar

- Si el cliente necesita controlar los datos que quiere obtener.
- Si la base de datos es muy grande, GraphQL permite consultar solo lo necesario.
- Se necesita reducir el over-fetching y under-fetching
- Cuando se necesita agregar datos de múltiples recursos.
- Si el sistema es complejo y los recursos dependen unos de otros
- Cuando el cliente tiene requisitos de consultas muy cambiantes

Implementaciones

Javascript	Python
Apollo Server	Ariadne

gRPC



Inicialmente era una versión de RPC (Remote Procedure Calls) desarrollada por Google (gRPC) pero actualmente es de código abierto. Requiere que cliente y servidor establezcan un contrato para comunicarse.

Otros protocolos como SOAP y REST usan XML y JSON para transmitir los mensajes, gRPC usa Protocol Buffers para serializar los datos y HTTP/2 como protocolo de transporte, esto hace que las llamadas sean mas rápidas y eficientes, ya que hace uso de multiplexación, compresión de encabezados y conexiones persistentes y permite streaming Bidireccional

Su propósito es ser una alternativa a lo métodos tradicionales como Rest, pero eficiente, escalable y de alto rendimiento, es muy útil en arquitecturas basadas en microservicios.

Cuando usar gRPC

- Cuando se requiere alto desempeño.
- Para comunicación entre dispositivos con pocos recursos (dispositivos IoT)
- Para comunicación en una arquitectura de microservicios

Implementaciones

Javascript	Python
gRPC-JS (Node) gRPC-WEB	grpcio

SOAP vs REST

Cuándo es mejor usar SOAP en lugar de REST?
Aunque REST es muy ligero y flexible, en ciertos escenarios SOAP es preferible sobre REST:
Sobre todo cuando se requieren reglas estrictas y funcionalidades avanzadas Somo seguridad.

Seguridad:
Cuando se requiere seguridad robusta, por ejemplo en aplicaciones bancarias o gubernamentales
donde la autenticación y encriptado es muy importante.

Transacciones complejas:
SOAP soporta transacciones distribuidas, lo que es útil si trabajas con operaciones que requieren
atomicidad, como varias transferencias bancarias en una sola transacción.

Integración entre sistemas empresariales:
SOAP es ideal y muy usado para integrarse con sistemas heredados o aplicaciones empresariales de misión critica
al hacer uso de estándares mundialmente aceptados (HTTP, XML, XSD, WSDL, WS-Security).

Comunicación asíncrona y confiable:
Si se necesita garantizar la entrega de mensajes incluso en caso de fallos en la red
(gracias a WS-ReliableMessaging), SOAP tiene ventajas significativas.

Compatibilidad con estándares establecidos:
SOAP está diseñado para adherirse a estándares establecidos y tiene soporte
para funcionalidades avanzadas como mensajes binarios y uso de diferentes protocolos
de transporte (no solo HTTP, como en REST).

REST vs GraphQL

Cuándo es mejor usar GraphQL en lugar de REST

REST es muy fáci de implementar y consumir pero en ciertos escenarios puede no ser lo mas óptimo sobretodo cuando lo que se necesita es:

Flexibilidad y precision
GraphQL al ser un lenguaje de consulta, nos permite especificar exactamente los datos que queremos obtener caso contrario de REST en el que cada endpoint expone una o recurso con una estructura ya predeterminada

Rendimiento
Mismo caso que el punto anterior, en ocasiones en REST para obtener datos de mas de un recurso, tendremos que hacer varias peticiones, en GraphQL en la misma petición podemos obtener todo.

Cliente con requerimientos cambiantes
Si parte cliente del sistema (FrontEnd) cambia frecuentemente los datos que necesita en REST, tendremos que re-programar nuestra API, en GraphQL esto no sera necesario ya que el cliente tiene la flexibilidad de especificar sus necesidades cambiaste de datos en la consulta.

HTTP Request, Long Polling , SSE, WebSockets, WebHooks

Aspecto a considerar	Http Req.	Long Pooling	SSE	WebSocket	WebHook
Sentido de la comunicación:	Unidireccional	Unidireccional	Unidireccional	Bidireccional	Unidireccional
Intercambio de datos:	Petición Respuesta	Petición queda abierta hasta recibir respuesta	Constante En tiempo real	Constante En tiempo real	Breve y esporádico
Si se requiere:	Solicitudes esporádicas	Cuando no es posible usar SSE o Websockets	Notificaciones en tiempo real	Intercambio constate con retraso mínimo	Reaccionar a eventos esporádicos